WEEK 6

Counters & Shift Registers

จาก Counter สู่ Serial Communication

Up/Down · Mod-N · Clock & PLL · PWM · Timer · SISO/SIPO/PISO/PIPO · UART/SPI · LFSR

AGENDA

What We'll Cover Today

01

Counters

Up/Down Counter, Enable, Mod-N, Ring, Johnson — รากฐานของ timer และ divider.

02

Clock & PLL

Clock properties, Jitter, Skew, PLL theory, การคำนวณ f_out = f_in × N/(M×K).

03

Counter Applications

Timer, Watchdog, PWM, Frequency Meter, Address Generator, Gray Code.

04

Shift Registers

SISO, SIPO, PISO, PIPO, Universal Shift Register และการใช้งานจริง.

05

Serial I/O & LFSR

UART, SPI, LFSR, Barrel Shifter — จาก shift register สู่งานอุตสาหกรรม.

1 · CONCEPT

What is a Counter?

Sequential circuit ที่ไล่ค่าตามขอบ clock — ใช้งานทุกที่ตั้งแต่นาฬิกาดิจิทัลจนถึง CPU pipeline

Counter = Register + Incrementer (หรือ Decrementer)

ทุก positive edge ของ clock → ค่าใน register จะถูกแทนที่ด้วยค่าถัดไปที่คำนวณจาก combinational logic

ถ้า next = current + 1 → Up Counter | ถ้า next = current − 1 → Down Counter

Timer / Watchdog

นับ clock cycle แล้ว timeout → interrupt

Clock Divider

แบ่ง CLOCK_50 MHz เป็น 1 Hz สำหรับตาคน

Address Generator

สร้าง address อ่าน ROM/RAM แบบ sequential

FSM State Encoding

Binary counter เป็น state register ของ FSM

1 · CONCEPT

4-bit Binary Up Counter — Timing Diagram

ค่าเพิ่มขึ้นทีละ 1 ทุก rising edge ของ clock: 0000 → 0001 → 0010 → … → 1111 → 0000 (wrap)

Clock Edge	count[3:0]	Decimal	Next
Rising edge 1	0000	0	0001
Rising edge 2	0001	1	0010
Rising edge 3	0010	2	0011
Rising edge 4	0011	3	0100
…	…	…	…
Rising edge 15	1110	14	1111
Rising edge 16	1111	15	0000 (wrap)

💡 4-bit counter นับได้ $2^4 = 16$ ค่า (0–15) แล้ว wrap กลับ 0 — เรียกว่า mod-16 counter

2 · CODE

4-bit Up Counter — Verilog

Synchronous reset, posedge clk

VERILOG

```
module counter_up (
  input  wire       clk,
  input  wire       rst_n,
  output reg  [3:0] count
);

always @(posedge clk) begin
  if (!rst_n)
    count <= 4'b0000;
  else
    count <= count + 1;
  end
endmodule
```

rst_n (active-low)

KEY[0] บน DE0 เป็น active-low → กด = 0 → reset

count <= count + 1

non-blocking (<=) เพราะอยู่ใน sequential always block

Overflow / Wrap

4-bit: 1111 + 1 = 0000 อัตโนมัติ ไม่ต้องเขียน if

2 · CODE

Counter with Enable

enable = 1 → นับ | enable = 0 → หยุดนับ (ค้างค่าเดิม)

VERILOG

```
module counter_en (
  input  wire       clk,
  input  wire       rst_n,
  input  wire       enable,
  output reg  [3:0] count
);

always @(posedge clk) begin
if (!rst_n)
    count <= 4'b0;

else if (enable)
    count <= count + 1;
  // else: count keeps its value
end
endmodule
```

Priority ของ if-else

1. rst_n (สูงสุด — reset ก่อนเสมอ)
2. enable (ถ้าไม่ reset → ดู enable)
3. else implicit (ค้างค่าเดิม)

ลำดับ priority นี้สำคัญมาก — สลับลำดับ
อาจทำให้ counter reset ไม่ได้

2 · CODE

Up/Down Counter

ใช้สัญญาณ dir ควบคุมทิศทาง: dir=1 → นับขึ้น, dir=0 → นับลง

VERILOG

```
module counter_updown (
  input  wire       clk, rst_n,
  input  wire       dir,  // 1=up, 0=down
  output reg  [3:0] count
);

always @(posedge clk) begin
  if (!rst_n)
    count <= 4'b0;
  else if (dir)
    count <= count + 1;
  else
    count <= count - 1;
  end
endmodule
```

DE0 Pin Mapping

```
clk    →  CLOCK_50
rst_n  →  KEY[0]
dir    →  SW[0]   (up/down)
count  →  LEDG[3:0]
```

Underflow Behavior

0000 − 1 = 1111 (wrap)

เหมือน overflow แต่ทิศตรงข้าม

ถ้าไม่ต้องการ wrap → เพิ่ม

if (count == 0) ป้องกัน

3 · ADVANCED

Ring Counter & Johnson Counter

Counter พิเศษ — ใช้ shift register เป็น counter แทนการบวก 1

Ring Counter (4-bit)

1000 → 0100 → 0010 → 0001 → 1000

Feedback: MSB → LSB (circular shift)
State = N (N-bit → N states)

ข้อดี: decode ง่าย (ดูแค่ 1 bit)

ข้อเสีย: ใช้ flip-flop มาก

หรือ 4 bits → 4 states (ไม่ใช่ 16)

Johnson Counter (4-bit)

0000 → 1000 → 1100 → 1110
→ 1111 → 0111 → 0011 → 0001 → 0000

Feedback: ~MSB → LSB (inverted)
States = 2N (4-bit → 8 states)

ข้อดี: glitch-free decode (2 bits)

ข้อเสีย: unused states → ต้อง

มี self-correcting logic

Gray Code Counter

เปลี่ยนทีละ 1 bit ต่อ transition — ป้องกัน glitch เมื่อข้าม clock domain

Decimal	Binary	Gray Code	bit ที่เปลี่ยน
0	0000	0000	—
1	0001	0001	bit 0
2	0010	0011	bit 1
3	0011	0010	bit 0
4	0100	0110	bit 2
5	0101	0111	bit 0
6	0110	0101	bit 1
7	0111	0100	bit 0

ปัญหาของ Binary

3 → 4 คือ 0011 → 0100
เปลี่ยน 3 bits พร้อมกัน!

ถ้าอ่านตอนกำลังเปลี่ยน
อาจได้ 0111 หรือ 0000
→ ค่าผิดสนิท

ใช้จริงที่ไหน

- Rotary/absolute encoder
- Async FIFO pointer (ข้าม clock domain)
- ADC output encoding
- Karnaugh map ordering

```
assign gray = bin ^ (bin >> 1);   ← แปลง Binary เป็น Gray ด้วย 1 บรรทัด
```

3 · ADVANCED

Counter Comparison

เลือก counter ที่เหมาะกับงาน — ไม่มีตัวไหนดีที่สุดสำหรับทุกกรณี

Counter Type	States (N-bit)	Bit Width	Speed	Use Case
Binary Up	2^N	N bits	เร็ว	General purpose, timer
Mod-N	N (arbitrary)	$\lceil \log_2 N \rceil$	เร็ว	Clock divider, BCD
Ring	N	N bits	เร็วมาก	One-hot FSM, sequencer
Johnson	2N	N bits	เร็วมาก	Glitch-free decode

ต้องการนับค่าทั่วไป?	→ **Binary Up Counter**	ง่ายที่สุด ใช้ได้เกือบทุกงาน
ต้องการ wrap ที่ค่าเฉพาะ?	→ **Mod-N Counter**	เช่น นับ 0–9 (BCD) หรือ 0–59 (วินาที)
ข้ามไปคนละ clock domain?	→ **Gray Code Counter**	เปลี่ยนทีละ bit → ปลอดภัยกว่า

💡 Ring และ Johnson ใช้ในงานเฉพาะทาง — เมื่อต้องการ decode state เร็วมากโดยไม่ต้องผ่าน logic

3 · ADVANCED

Clock Divider — จาก 50 MHz สู่ 1 Hz

CLOCK_50 เร็วเกินไปสำหรับตาคน — ต้อง "หาร" ความถี่ด้วย counter

VERILOG

```
module clk_divider (
  input  wire  clk_50m,
  input  wire  rst_n,
  output reg   clk_1hz
);

reg [24:0] divider;

always @(posedge clk_50m) begin

  if (!rst_n) begin
      divider <= 0;
      clk_1hz <= 0;
  end else if (divider == 24_999_999) begin
      divider <= 0;
      clk_1hz <= ~clk_1hz;
  end else
      divider <= divider + 1;
  end
endmodule
```

คำนวณ Divider

50,000,000 Hz ÷ 2 = 25,000,000

toggle ทุก 25,000,000 clock cycles
→ half-period = 0.5 sec
→ full period = 1 sec = 1 Hz

25-bit counter พอ (2^{25} = 33.5M > 25M)

ตัวอย่างอื่น (toggle count = f / 2×f_target)

10 Hz → toggle ทุก 2,500,000

1 kHz → toggle ทุก 25,000

100 Hz → toggle ทุก 250,000 (stopwatch)

3 · ADVANCED

PLL Phase-Locked Loop

ลองคำนวณทีละ step: ต้องการ 108 MHz สำหรับ VGA จาก 50 MHz

1. เป้าหมาย: f_out = 108 MHz, f_in = 50 MHz
2. หา ratio: 108 / 50 = 2.16 = 54/25 → ดังนั้น N = 54, M = 25
3. เช็ค f_ Voltage-Controlled Oscillator(VCO): f_in × N / M = 50 × 54/25 = 108 MHz → ต่ำกว่า VCO min (300 MHz), DE0 (300–600 MHz)
4. แก้: เพิ่ม output divider K=5 → ตั้ง N=270, M=25 → f_VCO = 50×270/25 = 540 MHz ✓ (300–600)
5. f_out = f_VCO / K = 540 / 5 = 108 MHz ✓ ← ได้ตามเป้าหมาย!

สูตรเต็ม: f_out = (f_in × N) / (M × K)

1. หา N/M จาก f_out/f_in (ทำเป็นเศษส่วน) 2. เช็ค f_VCO = f_in × N/M อยู่ใน 300–600 MHz 3. ถ้าไม่อยู่ → คูณ N และใช้ K หาร

PLL Calculation — ตัวอย่างเพิ่มเติม

f_out = (f_in × N) / (M × K) | Cyclone III VCO: 300–600 MHz

f_in	ต้องการ f_out	M (÷)	N (×)	f_VCO	K (÷)	f_out จริง
50 MHz	100 MHz	1	6	300 MHz	3	**100 MHz**
50 MHz	25 MHz	1	6	300 MHz	12	**25 MHz**
50 MHz	40 MHz	5	32	320 MHz	8	**40 MHz**
50 MHz	40 MHz	1	8	400 MHz	10	**40 MHz**
50 MHz	108 MHz (VGA)	5	54	540 MHz	5	**108 MHz**
50 MHz	65 MHz (XGA)	2	13	325 MHz	5	**65 MHz**

เลือกค่ายังไง?

1. หา f_VCO ที่หารด้วย f_out ลงตัว และอยู่ใน 300–600
2. K = f_VCO / f_out
3. N/M = f_VCO / f_in (ทำเป็นเศษส่วนอย่างต่ำ)

3 · HARDWARE

PLL on Cyclone III — ALTPLL IP

DE0 Cyclone III EP3C16F484 มี 4 PLL — ใช้ผ่าน Quartus IP Catalog

1. Tools → IP Catalog → Basic Functions → Clocks → ALTPLL
2. ตั้งชื่อ instance (เช่น my_pll) → Set input clock = 50 MHz
3. เลือก output frequency ที่ต้องการ (เช่น 100 MHz, 25 MHz)
4. เลือก output clocks (สูงสุด 5 outputs พร้อมกัน, แต่ละ output คนละ freq ได้)
5. Finish → Quartus สร้าง .v wrapper → Instantiate ใน top-level module

VERILOG — ALTPLL INSTANTIATION

```
wire pll_100m, pll_25m, locked;
my_pll pll0 (.inclk0(CLOCK_50), .c0(pll_100m), .c1(pll_25m), .locked(locked));
```

3 · ADVANCED

Counter Divider vs PLL

ทั้งคู่สร้าง clock ช้าลงได้ — แต่ PLL ทำได้มากกว่า

คุณสมบัติ	Counter Divider	PLL
ทิศทาง	หารได้อย่างเดียว (f ลดลง)	คูณและหารได้ (f เพิ่ม/ลด)
Duty Cycle	50% ถ้า toggle	ปรับได้ตามต้องการ
Jitter	สูง (gated clock)	ต่ำ (phase-locked)
ใช้ Logic Element	ใช้ LE/LUT	ใช้ dedicated PLL block
Clock Network	ไม่ใช้ global buffer	ใช้ global clock network
Multiple Outputs	ต้องสร้างหลาย divider	สูงสุด 5 outputs/PLL
Lock Time	ทันที	ต้องรอ lock (~µs)
	ck ง่าย ๆ	Production, high-speed, precise

Clock Jitter (ความสั่น)

เปรียบเหมือน: ครูตีระฆังทุกชั่วโมง
แต่บางทีเร็วไป 2 วิ บางทีช้าไป 3 วิ → คาบเรียนยาว/สั้นไม่เท่ากัน

ในวงจร: ขอบ clock มาไม่ตรงเวลา
ทุกครั้ง → timing margin ลดลง

5 · APPLICATIONS

Counter Applications — เอาไปทำอะไรได้บ้าง?

Counter ไม่ได้แค่ "นับเลข" — เป็นรากฐานของวงจรดิจิทัลแทบทุกชนิด

Mod-N Up

Timer / Watchdog

นับ clock จนครบเวลา
→ timeout signal

Free-run Up

PWM Generator

counter + comparator
→ ควบคุม duty cycle

Event Counter

Frequency Meter

นับ pulse ใน 1 วินาที
→ อ่านค่า Hz ตรง ๆ

Up + wrap

Address Generator

สร้าง address อ่าน
ROM/RAM ตามลำดับ

Mod-N Toggle

Clock Divider

หารความถี่ลงมา
ให้ตาคนเห็นได้

Binary/Gray

FSM State Register

เก็บ state ปัจจุบัน
ของ state machine

💡 ทุกตัวอย่างข้างบนใช้ counter ตัวเดียวกัน — ต่างกันแค่ "เอา output ไปทำอะไร" และ "wrap ที่ค่าไหน"

Timer & Watchdog Timer

นับ clock จนครบเวลาที่ตั้งไว้ → ส่งสัญญาณเตือน

VERILOG — PARAMETERIZED TIMER

```
module timer #(parameter PERIOD = 50_000_000)(
  input  wire clk, rst_n, start,
  output reg  timeout
);

reg [31:0] cnt;
reg running;

always @(posedge clk) begin
  if (!rst_n) begin
      cnt     <= 0;
      running <= 0;
      timeout <= 0;
  end else if (start) begin
      running <= 1;
      cnt     <= 0;
      timeout <= 0;
  end else if (running) begin
      if (cnt == PERIOD-1) begin
        timeout <= 1;
        running <= 0;
      end else
        cnt <= cnt + 1;
      end
end
endmodule
```

Watchdog Timer

Timer ที่ต้องถูก "kick" เป็นระยะ

ถ้าโปรแกรมค้าง → ไม่ได้ kick

→ timer ครบ → reset ทั้งระบบ

ใช้ในระบบที่ห้ามค้าง:

รถยนต์, การแพทย์, ดาวเทียม

ใช้จริงที่ไหน

- RTOS system tick (1 ms)
- Timeout ใน UART/SPI protocol
- Debounce ปุ่มกด (20 ms)
- Auto-sleep เมื่อไม่มีการใช้งาน
- Delay ใน state machine

PWM Generator — Counter + Comparator

หรี่ไฟ LED · ควบคุมความเร็วมอเตอร์ · ขับ servo — ทั้งหมดใช้ counter ตัวเดียว

VERILOG — 8-BIT PWM

```
module pwm (
   input  wire       clk, rst_n,
   input  wire [7:0] duty,
   output wire       pwm_out
);

reg [7:0] cnt;

always @(posedge clk)
   if (!rst_n)
      cnt <= 0;
   else
      cnt <= cnt + 1;  // free-run wrap
end

assign pwm_out = (cnt < duty);

endmodule
```

Duty % = duty / 256 × 100 **f_PWM = f_clk / 256**

duty=64 → 25% | duty=128 → 50% | duty=192 → 75%
50 MHz / 256 = 195 kHz (สูงพอที่ตาไม่เห็นกระพริบ)

หลักการทำงาน

counter นับ 0→255 วนไม่หยุด

ถ้า cnt < duty → output = 1

ถ้า cnt ≥ duty → output = 0

→ ยิ่ง duty สูง สัญญาณ HIGH นาน

→ LED สว่างขึ้น / มอเตอร์เร็วขึ้น

ใช้จริงที่ไหน

- หรี่ไฟ LED / backlight จอ
- ควบคุมความเร็วมอเตอร์ DC
- ขับ servo (pulse 1–2 ms)
- Class-D audio amplifier
- DAC ราคาถูก (PWM + RC filter)
- Switching power supply

Address Generator & Sequencer

Counter สร้าง address อ่าน memory ตามลำดับ — รากฐานของ CPU และ waveform generator

VERILOG — ROM ADDRESS GENERATOR

```
module addr_gen #(parameter DEPTH = 256)(
  input  wire       clk, rst_n, en,
  output reg  [7:0] addr
);

always @(posedge clk)

  if (!rst_n)
    addr <= 0;
  else if (en) begin
    if (addr == DEPTH-1)
      addr <= 0;
    else
      addr <= addr + 1;
End

endmodule
```

DDS Waveform Generator

counter → ROM ที่เก็บค่า sine
→ DAC → ได้คลื่นไซน์
เปลี่ยน step = เปลี่ยนความถี่

Program Counter (PC)

counter ชี้ instruction ถัดไป
ใน RISC-V: PC += 4 ทุก cycle
(จะได้เรียนละเอียด Week 10)

Video Scan Counter

H-counter นับ pixel ในแถว
V-counter นับแถว
→ address ของ frame buffer

What is a Shift Register?

Flip-Flop ต่ออนุกรมกัน — ข้อมูลเลื่อนทีละ 1 ตำแหน่งทุก clock edge

Shift Register = FF ต่อกันเป็นแถว โดย output ของตัวหนึ่งเป็น input ของตัวถัดไป

Counter คำนวณค่าถัดไปเอง (count + 1) · Shift Register แค่ "ย้าย" ข้อมูลที่มีอยู่ไปตำแหน่งข้างๆ

Clock	serial_in	q[3]	q[2]	q[1]	q[0]	อธิบาย
เริ่มต้น	—	0	0	0	0	register ว่าง
edge 1	1	1	0	0	0	1 เข้ามาที่ q[3]
edge 2	0	0	1	0	0	1 เลื่อนไป q[2]
edge 3	1	1	0	1	0	ข้อมูลเลื่อนต่อ
edge 4	1	1	1	0	1	ครบ 4 bit = 1011

ทุก protocol แบบ serial (UART, SPI, I2C, JTAG, USB) ใช้ shift register เป็นหัวใจในการส่ง/รับข้อมูล

Shift Register — เอาไปทำอะไรได้บ้าง?

ไม่ได้แค่ "เลื่อนบิต" — มันคือสะพานระหว่างโลก parallel กับ serial

PISO + SIPO - Parallel In, Serial Out + SIPO-Serial In, Parallel Out

Serial Communication

UART · SPI · I2C
ส่ง/รับทีละ bit ผ่านสายเส้นเดียว

SIPO-Serial In, Parallel Out

I/O Expansion

ใช้ 3 ขา MCU คุม LED ได้ 8–32 ดวง
(ชิป 74HC595)

SISO-Serial In, Serial Out

Delay Line / Buffer

หน่วงสัญญาณ N clock
ใช้ใน pipeline, CDC sync

PISO-Parallel In, Serial Out

Data Serialization

parallel bus → serial link
ลดจำนวนสายบน PCB

LFSR-Linear Feedback Shift Register

Pseudo-Random Gen

เลขสุ่ม · CRC · scrambler
test pattern (BIST)

SIPO-Serial In, Parallel Out

Pattern Detection

เก็บ N bit ล่าสุด แล้วเทียบ pattern
→ FSM sequence detector (Week 8)

💡 สังเกต: ทุกงานข้างบนใช้ shift register ตัวเดียวกัน — ต่างกันแค่ "ข้อมูลเข้าทางไหน ออกทางไหน"

4 ประเภท: SISO / SIPO / PISO / PIPO

แบ่งตามวิธีที่ข้อมูล "เข้า" และ "ออก" — serial (ทีละ bit) หรือ parallel (พร้อมกัน)

SISO

Serial In · Serial Out



เข้าทีละ bit ออกทีละ bit

ใช้: delay line, buffer
หน่วงสัญญาณ N clock

SIPO

Serial In · Parallel Out



เข้าทีละ bit ออกพร้อมกัน

ใช้: UART RX, 74HC595
รับ serial → ใช้งาน parallel

PISO

Parallel In · Serial Out



เข้าพร้อมกัน ออกทีละ bit

ใช้: UART TX, SPI master
ส่งออกทางสายเดียว

PIPO

Parallel In · Parallel Out



เข้าพร้อมกัน ออกพร้อมกัน

ก็คือ register ธรรมดา
(ที่เรียนไป Week 5)

→ = serial (ทีละ bit, ใช้ 1 สาย, ต้อง N clock) ⇉ = parallel (พร้อมกันทุก bit, ใช้ N สาย, 1 clock)

💡 SIPO และ PISO ใช้คู่กันเสมอในการสื่อสาร — ฝั่งส่งใช้ PISO, ฝั่งรับใช้ SIPO

6 · CODE

Shift Register — Verilog

ใช้ concatenation operator { } ในการเลื่อนบิต — เขียนได้บรรทัดเดียว

VERILOG — 8-BIT SHIFT REGISTER

```
module shift_reg (
  input  wire       clk, rst_n,
  input  wire       ser_in,
  output reg  [7:0] q
);

always @(posedge clk) begin
  if (!rst_n)
    q <= 8'b0;
  else
    // Shift LEFT: bit ใหม่เข้าที่ LSB
    q <= {q[6:0], ser_in};
  end
endmodule

// แบบอื่นๆ:

// Shift RIGHT: q <= {ser_in, q[7:1]};

// Rotate LEFT: q <= {q[6:0], q[7]};

// Rotate RIGHT: q <= {q[0], q[7:1]};
```

เข้าใจ { } — Concatenation

{ } = เอา bit มาต่อกันเป็นก้อนใหม่

q = 8'b1010_1100, ser_in = 1

q[6:0] = 010_1100 (ตัด MSB ทิ้ง)
{q[6:0], ser_in} = 0101_1001

→ ทุก bit เลื่อนซ้าย 1 ตำแหน่ง
→ bit ใหม่เข้าที่ตำแหน่งขวาสุด

⚠ ระวัง bit order! q[6:0] คือตัด bit บนสุดทิ้ง — ถ้าเขียน q[7:1] จะได้ shift ขวาแทน

Shift vs Rotate

Shift = ทิ้ง bit ที่หลุดออก | Rotate = วนกลับเข้าอีกฝั่ง

Universal Shift Register

ทำได้ทุกอย่างด้วย mode control 2-bit — คือชิป 74HC194 ที่ใช้ในอุตสาหกรรม

VERILOG — UNIVERSAL SHIFT REGISTER

```
module univ_shift (
  input  wire       clk, rst_n,
  input  wire [1:0] mode,
  input  wire       ser_l, ser_r,
  input  wire [7:0] par_in,
  output reg  [7:0] q
);

always @(posedge clk) begin

  if (!rst_n)
    q <= 8'b0;
  else
    case (mode)
      2'b00: q <= q;                    // Hold
      2'b01: q <= {ser_r, q[7:1]};      // Right
      2'b10: q <= {q[6:0], ser_l};      // Left
      2'b11: q <= par_in;               // Load
    endcase
  end
endmodule
```

mode	การทำงาน	ใช้เมื่อไร
00	Hold	หยุดค้างค่า
01	Shift Right	รับ/ส่ง LSB first
10	Shift Left	รับ/ส่ง MSB first
11	Parallel Load	โหลดข้อมูลใหม่

ทำไมต้อง Universal?

SPI ต้องส่งและรับพร้อมกัน
→ ใช้ mode 11 โหลดข้อมูลส่ง
→ ใช้ mode 10 เลื่อนออกทีละ bit
พร้อมรับ bit เข้ามาอีกฝั่ง
→ ครบ 8 clock = แลกข้อมูลเสร็จ

6 · APPLICATIONS

Serial ↔ Parallel Conversion

การใช้งานหลักที่สุดของ shift register — หัวใจของทุก communication protocol

ทำไมต้องแปลง? Parallel เร็ว แต่ใช้สายเยอะ (8 bit = 8 เส้น) · Serial ช้ากว่า แต่ใช้แค่ 1–2 เส้น

ยิ่งสายยาว ยิ่งแพงและเจอ noise → serial ชนะเสมอในการส่งระยะไกล (USB, Ethernet, HDMI ล้วนเป็น serial)

UART TX (PISO)

1. โหลด 8-bit data เข้า register
2. ส่ง start bit (0)
3. เลื่อนออกทีละ bit ตาม baud rate
4. ส่ง stop bit (1)

→ รวม 10 bit ต่อ 1 byte

UART RX (SIPO)

1. รอ start bit (ขอบขาลง)
2. sample ตรงกลางแต่ละ bit
3. เลื่อนเข้า register ทีละ bit
4. ครบ 8 bit → ยก data ออก

→ ต้อง sync baud rate ให้ตรง

SPI (Universal)

master + slave มี shift reg คนละตัว
ต่อกันเป็นวงกลม (MOSI/MISO)
ทุก clock: ส่งออก 1 bit + รับเข้า 1 bit
ครบ 8 clock → แลกข้อมูลกันเสร็จ
→ full-duplex ในตัว

💡 Week 14 จะได้สร้าง UART peripheral จริงบน SoC — พื้นฐานคือ shift register ที่เรียนวันนี้

LFSR — Linear Feedback Shift Register

Shift register + XOR feedback = เครื่องสร้างเลขสุ่มเทียม (pseudo-random)

VERILOG — 4-BIT LFSR

```
module lfsr_4bit (
  input  wire       clk, rst_n,
  output reg  [3:0] lfsr
);

wire fb = lfsr[3] ^ lfsr[2];  // taps

always @(posedge clk)
  if (!rst_n)
     lfsr <= 4'b0001;
  else
     lfsr <= {lfsr[2:0], fb};
endmodule
```

Width	Tap Positions	Sequence Length
4-bit	[4, 3]	15
8-bit	[8, 6, 5, 4]	255
16-bit	[16, 15, 13, 4]	65,535
32-bit	[32, 22, 2, 1]	4.29 พันล้าน

Sequence (15 states, ไม่ผ่าน 0000)

0001 → 0010 → 0100 → 1001 → 0011
→ 0110 → 1101 → 1010 → 0101 → 1011
→ 0111 → 1111 → 1110 → 1100 → 1000 → (วนกลับ 0001)

ใช้จริงที่ไหน

- CRC checksum (ตรวจข้อมูลผิดพลาด)
- Data scrambler (Ethernet, PCIe)
- BIST — ทดสอบชิปด้วยตัวเอง
- Test pattern generator
- Spread-spectrum clock (ลด EMI)
- Stream cipher อย่างง่าย

6 · APPLICATIONS

Shift Register ในงานจริง / อุตสาหกรรม

4 ตัวอย่างที่คุณเจอทุกวันโดยไม่รู้ตัว

74HC595 — ชิปยอดนิยม

SIPO 8-bit ราคาไม่กี่บาท · ใช้ 3 สาย (data, clock, latch) ได้ output 8 ขา
ต่อ cascade ได้ไม่จำกัด: 8 → 16 → 24 bits · ใช้ใน LED matrix, relay board, 7-seg หลายหลัก

JTAG Boundary Scan

ทุก FPGA/CPU มี shift register ล้อมรอบทุกขา I/O
เลื่อนค่า test เข้าไปตรวจว่าขาบัดกรีติดไหม โดยไม่ต้องแตะ PCB · USB-Blaster ที่ใช้โปรแกรม DE0 ก็คือ JTAG

FIR Digital Filter

Shift register เก็บ sample ย้อนหลัง N ตัว (tap delay line)
$y[n] = h_0 \cdot x[n] + h_1 \cdot x[n-1] + h_2 \cdot x[n-2] + \ldots$ · ใช้ใน audio DSP, sensor filter, AI accelerator (Week 15)

SERDES — Video/High-Speed Link

แปลง pixel data parallel → serial สำหรับ HDMI, DisplayPort, PCIe
ความเร็วระดับ Gbps ใช้ dedicated hardware block ใน FPGA (ไม่ใช่ LUT ธรรมดา)

Counter vs Shift Register — เลือกใช้อะไร?

ทั้งคู่สร้างจาก flip-flop เหมือนกัน แต่ทำงานคนละแบบ

ประเด็น	Counter	Shift Register
การทำงาน	คำนวณค่าถัดไปเอง (+1 / −1)	ย้ายข้อมูลที่มีอยู่ไปข้างๆ
Logic ที่ใช้	Adder / Incrementer	แค่ต่อสาย (wire) + FF
Resource	ใช้ LUT มากกว่า (มี adder)	ใช้ LUT น้อยมาก
ความเร็ว	ช้ากว่า (carry chain)	เร็วมาก (ไม่มี logic คั่น)
จำนวน State (N-bit)	2^N (นับได้ครบทุกค่า)	ขึ้นกับข้อมูลที่ป้อนเข้า
ใช้ทำอะไร	Timer, PWM, address, FSM state	Serial I/O, delay, LFSR, pattern
ตัวอย่างจริง	นาฬิกาจับเวลา, ควบคุมมอเตอร์	UART, SPI, 74HC595, CRC

💡 ในงานจริงมักใช้ทั้งคู่ร่วมกัน — เช่น UART ใช้ counter นับ baud rate + shift register ส่งข้อมูล

ใช้ Gemini ช่วยอะไรได้บ้าง?

AI ช่วยได้หลายขั้นตอน — แต่ต้องเข้าใจ output ก่อนนำไปใช้

"สร้าง 8-bit PWM module ที่ควบคุม duty ผ่าน input"

→ ได้ counter + comparator พร้อม comment อธิบายสูตร duty

"debug: shift register เลื่อนผิดทาง ข้อมูลออกทาง MSB"

→ Gemini ชี้ว่าสลับ q[6:0] กับ q[7:1] ใน concatenation

"อธิบายว่า LFSR taps [8,6,5,4] เลือกมาจากไหน"

→ อธิบาย primitive polynomial + ตาราง maximal-length

"เปรียบเทียบ SIPO กับ PISO ใช้ตอนไหนต่างกัน"

→ ตารางเทียบ + ตัวอย่าง UART TX/RX ที่เห็นภาพ

⚠ AI เขียนโค้ดที่ compile ผ่านได้ แต่ไม่รับประกันว่าตรงกับสิ่งที่คุณต้องการ — ต้อง simulate ตรวจสอบเสมอ

สรุปสัปดาห์ที่ 6

Counter = Register + Next-State Logic · Up/Down · Enable · Mod-N ✓

Clock: Frequency, Period, Duty Cycle, Jitter, Skew · PLL: $f_{out} = f_{in} \times N/(M \times K)$ ✓

Counter Applications: Timer, PWM, Frequency Meter, Address Gen, Gray Code ✓

Shift Register 4 แบบ: SISO · SIPO · PISO · PIPO ✓

Serial ↔ Parallel Conversion — หัวใจของ UART, SPI, I2C ✓

LFSR (pseudo-random, CRC) · Universal Shift Register · Barrel Shifter ✓

▶ Week 7: Finite State Machine — Part I (Moore FSM: Traffic Light Controller)